IMDB Review Sentiment Analysis: Mini-Project Report

Performed by: Gorelskii Roman

March 26, 2026

Abstract

This report presents an exploration of classical NLP pipelines for predicting the sentiment of IMDB movie reviews. Multiple preprocessing configurations combined with TF-IDF feature extraction and two classifiers (Logistic Regression and Multi-Layer Perceptron) were evaluate. Using 5-fold stratified cross-validation, performance across different preprocessing strategies was compared with feature importance analysis perfomed to understand model behavior depending on the strategy selected.

1 Project Overview

This project explores classical NLP pipelines for predicting the sentiment of IMDB movie reviews. The goal is to classify reviews as *positive* or *negative* based on their text. The dataset (1) is balanced for sentiment (positive/negative) and contains 5000 reviews (a subsample was chosen).

1.1 Pipeline Steps

1. **Preprocessing**: Tokenization (regex or NLTK), lowercasing, stopword removal, stemming/lemmatization.
2. **Feature Extraction**: TF-IDF vectorization (unigrams/bigrams).
3. **Classification**: Logistic Regression and Multi-Layer Perceptron (MLP).
4. **Evaluation**: 5-fold stratified cross-validation, reporting accuracy, F1, ROC-AUC, and PR-AUC.

2 Preprocessing Configurations

Multiple preprocessing configurations were tested, including:

- Regex vs. NLTK tokenization
- With/without stopword removal
- With/without stemming or lemmatization

3 Results Summary

Both Logistic Regression and MLP performed well, with Logistic Regression slightly outperforming MLP in most configurations. The best performance was achieved with `Regex + lower + stop` configuration for Logistic Regression (F1 = 0.8645) and `Regex + lower + stem` for MLP (F1 = 0.8544). Removing stopwords or applying stemming/lemmatization did not always improve results, suggesting that some stopwords or word forms may carry useful information for this task.

3.1 Key Metrics

Table 1: Logistic Regression performance metrics across preprocessing configurations

Configuration	**Accuracy**	**F1 Score**	**ROC-AUC**	**PR-AUC**
Majority baseline	0.4988 ± 0.0000	0.3328 ± 0.0000	0.5000 ± 0.0000	0.5013 ± 0.0000
Unigram TF-IDF	0.8428 ± 0.0099	0.8428 ± 0.0099	0.9215 ± 0.0102	0.9215 ± 0.0102
Regex + lower	0.8560 ± 0.0195	0.8560 ± 0.0195	0.9319 ± 0.0115	0.9321 ± 0.0110
Regex + lower + stop	$\mathbf{0.8646 \pm 0.0120}$	$\mathbf{0.8645 \pm 0.0120}$	$\mathbf{0.9375 \pm 0.0096}$	0.9363 ± 0.0098
Regex + lower + stem	0.8578 ± 0.0152	0.8578 ± 0.0152	0.9365 ± 0.0112	$\mathbf{0.9373 \pm 0.0102}$
Regex + lower + lemma	0.8544 ± 0.0145	0.8544 ± 0.0145	0.9320 ± 0.0114	0.9324 ± 0.0110
NLTK + lower	0.8548 ± 0.0197	0.8548 ± 0.0197	0.9278 ± 0.0122	0.9280 ± 0.0125
NLTK + lower + stop	0.8594 ± 0.0142	0.8593 ± 0.0142	0.9321 ± 0.0102	0.9304 ± 0.0115
NLTK + lower + stem	0.8552 ± 0.0124	0.8552 ± 0.0123	0.9324 ± 0.0115	0.9333 ± 0.0114

Table 2: Multi-Layer Perceptron (MLP) performance metrics across preprocessing configurations

Configuration	**Accuracy**	**F1 Score**	**ROC-AUC**	**PR-AUC**
Majority baseline	0.4988 ± 0.0000	0.3328 ± 0.0000	0.5000 ± 0.0000	0.5013 ± 0.0000
Unigram TF-IDF	0.8344 ± 0.0065	0.8344 ± 0.0065	0.9163 ± 0.0045	0.9159 ± 0.0081
Regex + lower	0.8518 ± 0.0130	0.8518 ± 0.0130	0.9303 ± 0.0080	0.9293 ± 0.0081
Regex + lower + stop	0.8522 ± 0.0138	0.8522 ± 0.0138	0.9282 ± 0.0089	0.9277 ± 0.0082
Regex + lower + stem	$\mathbf{0.8544 \pm 0.0128}$	$\mathbf{0.8544 \pm 0.0128}$	$\mathbf{0.9349 \pm 0.0090}$	$\mathbf{0.9345 \pm 0.0095}$
Regex + lower + lemma	0.8488 ± 0.0099	0.8488 ± 0.0099	0.9305 ± 0.0079	0.9288 ± 0.0093
NLTK + lower	0.8466 ± 0.0154	0.8466 ± 0.0154	0.9262 ± 0.0097	0.9242 ± 0.0096
NLTK + lower + stop	0.8482 ± 0.0128	0.8482 ± 0.0128	0.9233 ± 0.0090	0.9204 ± 0.0111
NLTK + lower + stem	0.8520 ± 0.0134	0.8520 ± 0.0134	0.9307 ± 0.0087	0.9292 ± 0.0096

4 Analysis

4.1 Tokenization

Regex and NLTK tokenizers performed similarly, but regex sometimes better captured domain-specific patterns (e.g., contractions, abbreviations). The best performing configuration used regex tokenization, achieving an F1 score of 0.8645.

4.2 Stopword Removal

Removing stopwords did not always help; the best overall performance (F1 = 0.8645) was achieved with stopword removal using regex tokenization. However, for MLP, the best configuration (F1 = 0.8544) was without stopword, suggesting that some stopwords may be informative in the sentiment/usefulness context depending on the model.

4.3 Stemming/Lemmatization

These techniques reduced vocabulary size but sometimes removed useful distinctions, leading to minor drops in performance compared to the best configuration. For Logistic Regression, stemming (F1 = 0.8578) and lemmatization (F1 = 0.8544) both performed slightly worse than the stopword removal configuration.

4.4 Models Comparison

Logistic Regression consistently outperformed MLP across all configurations, with the best LR configuration achieving F1 = 0.8645 compared to MLP's best of F1 = 0.8544. In the given setting the simpler linear model outperforms more complex MLP.

4.5 Feature Importance

To interpret the Logistic Regression models, the most influential features were analysed (words/phrases) for each preprocessing configuration. For each of them, the model was retrained on the full dataset, and the top 10 positive and negative features were selected and visualized.

The analysis was performed for all seven preprocessing configurations. For each, two bar plots were generated: one for the top positive features and one for the top negative features. This allows for direct comparison of which tokens are most influential under different preprocessing strategies.

4.5.1 Key Observations (Figure 1)

- Certain words consistently appear as strong predictors across configs, indicating robust signals in the data. Words like "great", "amazing", "excellent" were strong positive predictors, while "worst", "terrible", "awful" were strong negative predictors across configurations.
- The set of top features can shift depending on the selected preprocessing, highlighting the impact of it on model interpretability.
- Some configs (e.g., with stopword removal) may lose potentially informative words, while stemming/lemmatization can merge related forms.

5 Prospects for Future Work

- **Error Analysis**: Examine misclassified reviews to identify common error patterns.
- **Threshold Tuning**: Explore different probability thresholds for classification.
- **Advanced Features**: Experiment with word embeddings (Word2Vec, GloVe) or transformer-based models (BERT) for comparison.

6 Conclusion

Classical NLP pipelines with TF-IDF and simple classifiers can achieve strong results on text classification tasks like sentiment analysis. The best model (Logistic Regression with regex tokenization, lowercasing, and stopword removal) achieved an F1 score of 0.8645 and ROC-AUC of 0.9375. Careful preprocessing and model selection are important, but more complex pipelines do not always guarantee better performance. The results show that even simple linear models can effectively capture sentiment patterns in movie reviews.

References

[1] LAKSHMIPATHI N, "IMDB Dataset of 50K Movie Reviews" Kaggle, `https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews`

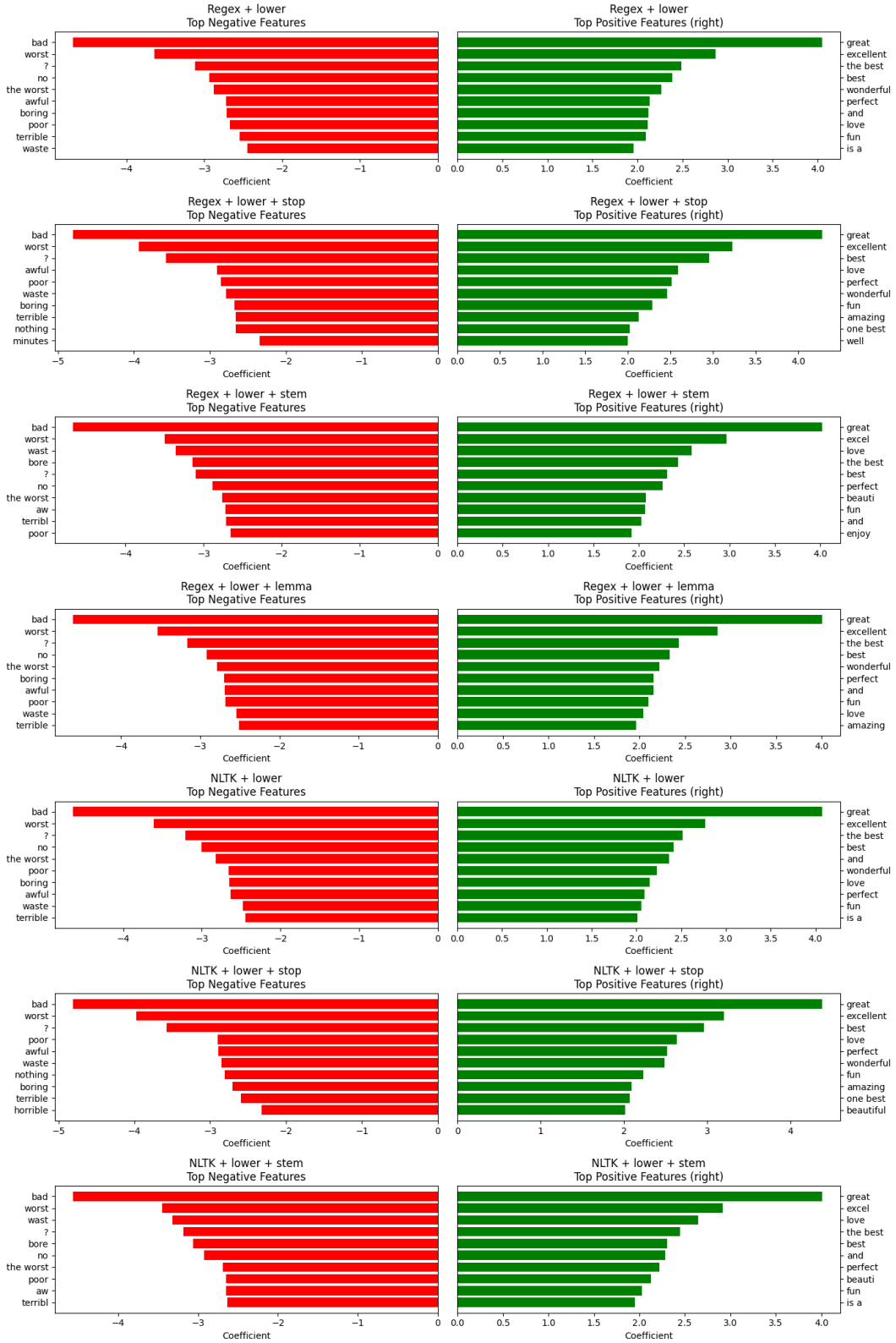

Figure 1: Feature importance